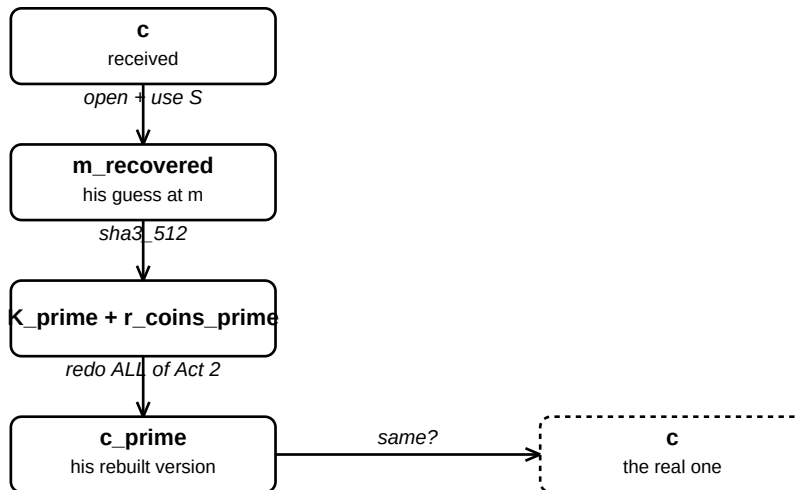


Act 3: Bob's Decapsulate

Bob receives c and nothing else. He has to do two separate jobs:

1. work out the shared secret
2. prove the ciphertext was built honestly, and not crafted by an attacker

Job 2 is the one that matters for security, and it takes most of the Act.



YES -> $K_{\text{final}} = K_{\text{prime}}$ (the real shared secret)

NO -> $K_{\text{final}} = \text{hash}(\text{zed} + c)$ (the imposter, quietly)

Act 3 step by step

Step 0 - open the ciphertext

```
u_received, v_received = ciphertext_opener(c)
```

WHAT: unpack c back into working numbers.

HOW: slice the blob apart, byte_decode each piece, then decompress.

IMPORTANT: what comes out is CLOSE to Alice's u and v but NOT identical. Compression threw detail away on the way out and it cannot be recovered. The code prints this explicitly and it shows False - which is correct and expected, not a bug.

Step 1 - recover the message

```
w = v_received - (S . u_received)
```

WHAT: undo the maths to get back to something close to m_encoded.

HOW: remember what v was built from:

```
v = b.r + e_two + m_encoded          and          b = A.S + e
```

so v contains (A.S).r hidden inside it. And $u = A_t.r + e_{one}$, so $S.u$ gives back roughly that same (A.S).r term. Subtracting one from the other cancels the big shared part and leaves m_encoded plus a small pile of leftover noise.

Bob is the only person who can do this, because it needs S.

Step 1b - turning w back into bits

```
zero_check = min(abs(w[i] - 0), abs(w[i] - q))
```

```
half_check = abs(w[i] - q // 2)
```

```
if half_check > zero_check: bit is 0    else: bit is 1
```

WHAT: decide, for each of the 256 positions, whether it was a 0 or a 1.

HOW: each number should be sitting near either 0 or near halfway (1665). Measure how far it is from each, and go with whichever is closer.

The min() on zero_check handles wraparound - a value like 3320 is only 9 away from 0 going the other way round, so it counts as near zero, not near the top.

THIS IS WHY THE NOISE CAN BE TOLERATED: the gap between 0 and 1665 is enormous.

The noise would have to stack up over 800 in one direction to push a value across the halfway line and flip the answer. The published odds of that happening for ML-KEM-1024 are about 2^{-174} .

Act 3 step by step (continued)

Step 2 - re-hash

```
g_output_bob = sha3_512(byte_encode(m_recovered, 1) + H(ek))
K_prime      = g_output_bob[0:32]
r_coins_prime = g_output_bob[32:64]
```

WHAT: exactly what Alice did in Act 2, but starting from `m_recovered`.

HOW: same hash, same split. Bob reuses his STORED `H(ek)` from Act 1 rather than recomputing it.

If `m_recovered` really is Alice's `m`, then `K_prime` will be identical to her `K`, because the same input through the same hash always gives the same output.

Step 3 - regenerate the noise

WHAT: rebuild `r'`, `e_one'`, `e_two'` from `r_coins_prime`.

HOW: the same position-tagged `shake_256` and CBD table trick. Nothing new.

This only works because Alice DERIVED her noise from her message rather than picking it independently. That decision back in Act 2 is what makes this whole check possible.

Step 4 - rebuild the ciphertext

```
u_prime = A_t . r' + e_one'
v_prime = b_t . r' + e_two' + m_recovered_encoded
```

WHAT: redo Alice's entire encryption, from Bob's side.

HOW: identical formulas. Bob uses his OWN `A` (he had the real one all along, no need to rebuild from `rho`) and his own `b`.

`m_recovered_encoded` uses the same $(q+1)//2 = 1665$ rounding Alice used. It has to match hers exactly or the rebuilt ciphertext would never line up.

Steps 5 and 6 - the integrity check

```
c_prime = ciphertext_maker(u_prime, v_prime)
ciphertext_matches = (c_prime == c)
```

WHAT: the actual security check of the whole scheme.

WHY NOT JUST COMPARE `u_prime` TO `u`: because compression is lossy, those would never match exactly even when everything is perfectly fine.

THE FIX: run `u_prime` and `v_prime` through the SAME compress-and-pack steps Alice used, producing Bob's own `c_prime`, then compare the finished bytes. Both sides go through identical lossy steps, and lossiness is deterministic - the same input always gives the same output - so an honest message produces byte-for-byte identical ciphertexts.

Why the check exists at all

It is tempting to think Bob only needs to recover a message and that is that. He does not, and the reason is worth being clear about.

Decoding never fails

Feed the decode maths ANY u and v - real, fake, complete garbage - and it will always hand back some 256-bit result. There is no such thing as a ciphertext that does not decode. So checking whether a message came out is a test that is always true, and therefore tests nothing at all.

What the check actually proves

Not that something decoded, but that the ciphertext was HONESTLY BUILT - by someone who genuinely knew the message and followed the whole process properly.

An attacker can craft a u and v pair that was never produced by encrypting anything, purely to probe Bob's secret S . That fake pair still decodes to some message. Without the rebuild-and-compare check, Bob would have no way of ever noticing, and repeated probes could slowly leak information about S .

Redoing the entire encryption and comparing is the one step that closes that off.

The two endings

PASS:

`K_final = K_prime`

The confirmed shared secret. Identical to Alice's K .

FAIL:

`imposter_protocol = zed + c`

`K_final = shake_256(imposter_protocol).digest(32)`

A decoy key, built from Bob's private zed and the ciphertext he actually received.

The decoy is a normal-looking 32 bytes and flows into Act 4 exactly like a real key. Nothing downstream knows or cares which branch produced it.

WHY BOTHER: if Bob visibly errored on a bad ciphertext, an attacker could send thousands of probes and learn from which ones failed. Because he behaves identically either way, there is nothing to observe. The rejection is silent.

The attacker cannot predict the decoy either, since zed is private. So a tampered message simply leads to Alice and Bob holding different keys - the conversation quietly fails and they would need to start a fresh exchange.

End of Act 3

Bob holds K_{final} . Nothing has been sent back to Alice - Decapsulate replies to nobody. Both sides now independently hold what should be the same 32 bytes, and that value was never transmitted in either direction.